

INTERNAL · PRODUCT & ENGINEERING

Mentable Institutional Platform

Product requirements and deployment plan for running mentorship programs inside universities, companies and communities, on the mentorship engine that is already in production.

Written against the live codebase and database, October 2026 · Schema state: migration 0030 applied · mentable.co

1. Executive summary

Mentable already runs real mentorship: 45 profiles, 10 active relationships, 42 messages, 38 sessions, 27 goals and 14 requests in production. The institutional work does not replace any of that. It adds a tenancy layer above it so one installation can run several named programs for several organizations, and gives program leads enough visibility to run a program without being able to read the conversations inside it.

Migration 0029 had already built most of the tenancy model to support Cohort 001. Migration 0030 therefore renames one table, adds four, and adds two columns, rather than introducing a parallel organizations system. The result is multi-tenant and verified isolated, with 26 automated security checks passing.

2. Production baseline

TABLE	ROWS	ROLE
profiles	45	26 mentors, 19 mentees, 2 demo
mentorships	10	All individual; all <code>cohort_id</code> IS NULL
messages	42	Private to the two participants
sessions	38	Scheduled and completed conversations
mentorship_goals	27	Shared goals
mentorship_requests	14	Mentee-initiated
action_items	2	Agreed next steps
notifications	31	In-product only

3. What migration 0029 already solved

The audit found that the institutional model largely existed under narrower names. Building `organizations`, `program_members` and `program_applications` as new tables would have duplicated deployed, working schema.

CONCEPT	EXISTING TABLE	DECISION
Tenant	<code>institutions</code>	Renamed to <code>organizations</code>
Program	<code>programs</code>	Reused unchanged
Program instance	<code>cohorts</code>	Reused unchanged
Membership	<code>cohort_memberships</code>	Reused unchanged
Applications	<code>cohort_applications</code>	Reused; already has <code>cohort_id</code>

Matching inputs	<code>career_pathways</code> , <code>mentor_profiles.can_help_with</code>	Reused
Token security	<code>activation_tokens</code>	Pattern reused for invitations

4. Architecture

`organizations` → `programs` → `cohorts` → `cohort_memberships` , with `mentorships` optionally scoped to a cohort. Cohort sits *above* mentorship: membership says who belongs, pairing stays where it already worked.

Backward compatibility

The guarantee: `mentorships.cohort_id IS NULL` means an ordinary Mentable relationship, behaving exactly as before. All 10 production mentorships are in this state and were not modified. `origin` defaults to `'requested'` , which is accurate for every existing row.

Three authorities, deliberately separate

AUTHORITY	SOURCE	SCOPE
Platform admin	<code>profiles.is_admin</code>	Mentable-wide. Applications, inquiries, reports.
Organization admin	<code>organization_members</code>	One organization. Roles: owner, admin, viewer.
Program admin	<code>program_admins</code>	Specific programs inside one organization.

None implies another. An organization owner is not a platform admin; a program lead for Family Business does not inherit the whole university. `is_admin` is not user-writable (migration 0015 revoked the column grant), so platform admin can only be granted by direct SQL.

Role and permission matrix

CAPABILITY	OWNER	ADMIN	VIEWER	PROGRAM ADMIN
See organization overview	✓	✓	✓	Their programs only
See cohort participants	✓	✓	✓	Their programs only
See engagement metrics	✓	✓	✓	Their programs only
Invite participants	✓	✓	—	Their programs only
Read private messages	—	—	—	—
Reach <code>/admin</code>	—	—	—	—

5. Database changes (migration 0030)

CHANGE	DETAIL
Rename	<code>institutions</code> → <code>organizations</code> ; <code>programs.institution_id</code> → <code>organization_id</code> . <code>ALTER TABLE RENAME</code> preserves oid, rows, indexes and FKs.
Column	<code>organizations.kind</code> — university, company, nonprofit, veterans, association, foundation, other
New	<code>organization_members</code> (org, profile, role; unique per pair)
New	<code>program_admins</code> (program, profile; unique per pair)
New	<code>program_invitations</code> (SHA-256 token, cohort, email, role, expiry, accepted/revoked)
New	<code>organization_inquiries</code> (public form submissions, status workflow)
Columns	<code>mentorships.cohort_id</code> (nullable FK), <code>mentorships.origin</code> (requested / suggested / admin_created)
Indexes	member-by-profile, admin-by-profile, invitation-by-cohort and by email, partial index on <code>mentorships.cohort_id</code>
Bootstrap	First platform admin set by email in the migration, resolving the zero-admin production gap

RLS strategy

All five new tables: RLS enabled, **zero policies, zero grants** to `anon` or `authenticated` . They are reachable only through service-role server routes that resolve the caller's membership first. This matches every table added since migration 0024 and is the reason cross-tenant isolation is enforceable rather than aspirational.

6. Privacy boundary

lib/org/metrics.ts never reads messages . Not filtered, not counted, not joined. Activity is derived from sessions, goals and action items — the structural record of a relationship rather than its content. An administrator can see that a relationship has gone quiet for 30 days without seeing a word of it. This is covered by an automated test that fails if the string `from('messages')` ever appears in that module.

No composite "engagement score" or "relationship health index" exists. Those compress several real facts into one number nobody can act on and invite comparing people. The metrics are counts of things that did or did not happen.

7. Matching

No separate matching engine and no `program_matches` table. An administrator creating a match is creating a `mentorship` ; the only thing the existing row could not express was where it came from, which `origin` now records.

- **Mentee request** — existing `mentorship_requests` flow, unchanged. **SHIPPED**
- **Admin-created** — mentorship inserted directly, `origin='admin_created'`. **SCHEMA READY, UI P1**
- **Suggested** — proposed pairing requiring acceptance. **P1**

8. Invitations

Reuses the activation-token contract already in production: 32 random bytes, stored only as SHA-256, single use, 14-day expiry, bound to the invited address. Supports `mentee`, `mentor` and `program_admin`, so delegating a program uses the same audited, expiring path as inviting a student rather than a second mechanism. Redemption at `/invite` writes membership first and burns the token last, so a mid-failure leaves a working link rather than a locked-out participant.

Delivery is manual. Mentable does not send participant mail. The admin UI states this on the screen rather than implying an email was sent.

9. Routes delivered

ROUTE	ACCESS	PURPOSE
<code>/organizations</code>	Public	Institutional landing page and inquiry form
<code>/organization</code>	Org member	Organizations this account administers
<code>/organization/[orgId]</code>	Org member	Programs, cohorts, rolled-up engagement
<code>/organization/[orgId]/cohort/[cohortId]</code>	Org/program admin	Participants, relationships, invitations
<code>/invite</code>	Token	Accept a program invitation
<code>/admin/inquiries</code>	Platform admin	Inbound institutional interest
POST <code>/api/org/invite</code>	Org/program admin	Mint an invitation
POST <code>/api/org/inquiry</code>	Public, throttled	Store an inquiry

10. Security verification

26 automated checks, all passing, against the live database:

- Anonymous clients denied (42501) on all seven institutional tables, read and write
- Org A admin cannot resolve access in Org B
- Viewer resolves but cannot mutate
- Delegated program admin is not org-wide and cannot reach a sibling program
- Cross-tenant cohort guard: a cohort in Org B does not resolve for an Org A admin
- Organization admin is not a platform admin; exactly one platform admin exists
- Invitations store only the hash; the raw token matches no row

- Used invitations cannot be replayed; expired invitations are detectable
- Metrics module provably never queries `messages`
- Anonymous clients cannot grant themselves organization membership

11. Priorities

ITEM	STATE
Tenancy schema, RLS, isolation	SHIPPED
Org admin: overview, cohort detail, engagement	SHIPPED
Invitations: mint, redeem, expire, revoke	SHIPPED
Public <code>/organizations</code> and inquiry flow	SHIPPED
Admin-created match UI	P0
Participant-facing program identity on dashboard	P0
Automated invitation and applicant email	P0
Create program / cohort from the UI	P1
Rematch, pause, remove participant	P1
Suggested matches	P1
Participant-reported outcome surveys	P1
Configurable application form builder	P2
CSV export, SSO, billing, CRM, calendar	P2

12. Deliberately not built

Form builder, SSO, billing, outcome surveys, AI matching, CRM and calendar integrations, advanced exports, custom branding, automated program communications, and nested organization hierarchies. None is blocked by the schema; each was left out because Mentable has no usage that would tell us the right shape for it.

13. Pilot deployment

1. Create the organization and set its first owner (SQL, two inserts)
2. Create a program and a cohort
3. Invite mentors and students; send links manually
4. Participants accept, landing in the existing mentorship product

5. Match by hand; relationships use the existing goals, sessions and action items

6. Program lead watches participation and momentum at `/organization`

Steps 1 and 2 are SQL today, which is acceptable for a first pilot where Mentable sets the program up, and is the first P1 to remove.

14. Risks

RISK	MITIGATION
Manual email delivery does not scale past a pilot	P0. Transactional email is the single largest operational gap.
Program setup requires engineering	P1 admin UI; acceptable while Mentable runs the first deployments.
Institution expects message visibility	Stated publicly on <code>/organizations</code> and in the admin UI. A boundary to sell, not apologise for.
Matching quality at scale	Manual matching is correct at pilot size and buys the data a real model would need.
Outcome attribution	Activity, engagement and self-reported outcomes are reported separately. Mentable does not claim to cause employment outcomes.

15. Production readiness

AREA	STATE
Multi-tenant isolation	Production ready — 26 checks passing
Privacy boundary	Production ready — enforced at the database
Org admin read surface	Production ready
Invitations	Pilot ready — delivery is manual
Program setup	Pilot ready — SQL, not UI
Matching	Pilot ready — by hand
Outcome measurement	Needs work — nothing collects self-reported outcomes yet
Billing, SSO, integrations	Not started, not blocked